VD notation

Can Valid

December 2025

1 Introduction

SDVD interface (ADT). State: a named authority $(\text{Name}(D), D)$ with an entry-log $(E_n)_{n\geq 0}$. Operations:

$$\begin{aligned} \text{lookup}(w) &:= D(w), \\ \text{define}(w,\delta)E_n &: \; w := \delta \text{ (append)}, \\ D^{(n)} &:= \text{stage}(n), \\ {}_n(w) &:= \exists k < n \text{ s.t. } \text{head}(E_k) = w. \end{aligned}$$

Scope of this paper. We treat the SDVD as an append-only construction: entries are added in a fixed order and no in-place edits are permitted. We therefore do not propose a governance theory of error-correction (renaming, deprecation, merge/split policies, etc.) in this work. If an author wishes to revise an entry, the intended procedure here is to restart the build from a clean log. A separate follow-up will study admissible update policies and their effect on recognisable law-formation.

Important clarifier We need to make sure readers understand that entries to the VD can be made even though the entries are not physical laws. An example might be "mass1 = the mass associated with particle 1." or "spring extension = the amount a spring is displaced by from its equilibrium position. "

SDVD (abstract interface). An SDVD instance consists of a name $\text{Name} \in \text{Strings}$, a map $D : \text{Strings} \to \text{Strings}$, and an append-only log $(E_n)_{n\geq 0}$.

Operations:

$$\begin{aligned} \text{lookup}(w) &:= D(w), \\ \text{define}(w,\delta)E_n &: \; w := \delta, \\ D^{(n)} &:= \text{stage}(n) \quad (\text{replay } E_0, \ldots, E_n), \\ {}_n(w) &:= \exists k < n \text{ with } \text{head}(E_k) = w, \\ \text{Triplet}(i,j,k) &:= \text{mutual token-mentions among } (E_i, E_j, E_k). \end{aligned}$$

Concept	Paper notation	Pseudocode	Python
Lookup	$D(w)$	`SDVD.D(w)`	`D(w)`
Commit entry	$E_n : w := \delta$	`SDVD.define(w,)`	`define(w,d)`
Entry-log view	$(E_n)_{n\geq 0}$	`SDVD.print_log()`	`print_log()`
Stage (snapshot)	$D^{(n)}$	`SDVD.stage(n)`	`stage(n)`
Previously defined	${}_n(w)$	`SDVD.previously_defined(w,before=n)`	`previously_defined(w,before)`
Triplet recogniser	(i, j, k)	`SDVD.is_triplet(i,j,k)`	`is_triplet(i,j,k)`

Table 1: Core SDVD operations. Pseudocode is shorthand for the formal interface.

We treat all recognition predicates (e.g. *Triplet*) as purely syntactic: they operate on strings.

Multi-valued lookup (polysemy) and why $D : \text{Strings} \to \text{Strings}$ is too strong. In the early presentation it is tempting to model a dictionary as a function $D : \text{Strings} \to \text{Strings}$, sending each word w to a unique definition-string $D(w)$. However, this single-valued model is incompatible with the intended SDVD workflow: (i) the construction is *append-only* (entries are added in a linear order), and (ii) a word may legitimately acquire multiple simultaneously active definition-strings (e.g. distinct senses, regimes, approximations, or competing proposals). If we insist on D being a function, then whenever a word is defined twice we must quietly introduce an additional policy ("latest definition wins", "first definition wins", "deprecated definitions are inactive", etc.). Such a policy is precisely the kind of dictionary-modification rule we aim to *avoid* in this paper.

Entry-log as primitive object. We therefore take the primitive object to be the entry-log rather than a single-valued map. An entry is a pair

$$E_n := (w_n, \delta_n) \in \text{Strings} \times \text{Strings},$$

and a stage-n dictionary state is the finite set (or ordered list)

$$\mathcal{D}^{(n)} := \{(w_k, \delta_k) : 0 \leq k \leq n\}.$$

Lookup is induced from the log as a *multi-map* (equivalently, a relation)

$$\mathcal{D}^{(n)}[w] := \{\delta \in \text{Strings} : (w, \delta) \in \mathcal{D}^{(n)}\} \subseteq \text{Strings},$$

or, when order matters, as the list of δ_k with headword $w_k \equiv w$ in increasing log index k.

Why this change is necessary (and not an extra rule). Allowing $\mathcal{D}^{(n)}[w]$ to contain multiple definition-strings does not add a new governance rule; it removes an implicit one. The append-only log already determines *what commitments have been made* (the multiset of entries), and multi-valued lookup

simply reports those commitments without choosing between them. Any subsequent practice of selecting, ranking, or reconciling definitions—for example, a choice function $S^{(n)}(w) \in \mathcal{D}^{(n)}[w]$ or a consistency filter—is a separate layer of methodology, which we postpone.

Consequences for "previously defined" and law-recognition. With the log as primitive, "previously defined" is a statement about headwords in the log:

$$_n(w) \ :\Longleftrightarrow \ \exists\, k < n \text{ such that } w_k \equiv w.$$

Law-recognition likewise remains entry-indexed: a triplet is a property of three entries (E_i, E_j, E_k), not of a globally unique value $D(w)$. This keeps the formal system faithful to the guiding principle that SDVD commitments are syntactic (strings in an ordered log), while deferring semantic questions of consistency or preferred usage to later work.

Unique-November constraint (one November word per law). In addition to the entry-log and triplet recognition rules, we impose a simple uniqueness constraint on *law anchors*. Each recognised physical law has a designated *November* headword (the previously-logged term that is locked to two newly introduced terms). We require that no headword may serve as the November word for more than one law.

Formally, let $\mathcal{L}$ denote the set of recognised law-events (triplets), and let $\mathrm{Nov} : \mathcal{L} \to \mathrm{Strings}$ return the November headword of a law-event. The *unique-November* constraint is

$$\forall w \in \mathrm{Strings}, \qquad \big|\{\, L \in \mathcal{L} : \mathrm{Nov}(L) \equiv w \,\}\big| \leq 1,$$

i.e. Nov is injective. This constraint does *not* prohibit polysemy: a headword may still admit multiple simultaneously active definition-strings in the entry-log. It only prohibits multiple distinct *law-introductions* from being anchored to the same November headword.

Hypergraph interpretation. Represent each law-event $L \in \mathcal{L}$ as a hyperedge incident to the participating headwords, and mark one incident vertex as the November vertex. The unique-November constraint is equivalent to stating that the November-marked incidence relation forms a matching: each word-vertex has November-degree at most one. This endows the law-subgraph with a clean provenance structure (a unique "anchor" per law) while leaving the full entry-log free to contain additional, non-law definitions of the same headwords.

We will most likely also need to write procedures to police this new rule when in code.